

CSI4107 Assignment 2

Henjing Zhang 300288003

Tom Cui 300345709

Zhiyu Lin 300255509

Work distribution

Hengjing Zhang, Tom Cui

Baseline System & Indexing:

- Implemented the inverted index and TF-IDF/BM25 ranking functions.
- Ensured that the BM25 results are generated correctly so that they can be used as input for the hybrid re-ranking.

Zhiyu Lin, Tom Cui

Neural Models and Re-Ranking Module:

- Developed the neural language model integration by implementing the modules for BERT and the Universal Sentence Encoder.
- Designed and implemented the vectorized cosine similarity functions and neural re-ranking functions in `neural_ranking.py`.
- Worked on integrating neural embedding generation and ensuring that both BERT and USE re-ranking methods yield MAP and P@10.

Hengjing Zhang, Zhiyu Lin

System Integration & Performance Optimization:

- Updated the `main.py` for Assignment 2 that combines the baseline results with the neural re-ranking outputs.
- Implemented the performance improvements so that the overall running time is reduced and visual feedback is provided.
- Coordinated the integration of all components, ensured that the system produces the correct output files, and performed the `trec_eval` for comparison with Assignment 1.

Functionality Overview

This project implements a hybrid information retrieval system that re-ranks `BM25` baseline results using neural embeddings from two models:

1. **BERT** (`bert_model.py`)
2. **Universal Sentence Encoder (USE)** (`use_model.py`)

It performs the following tasks:

- Loads queries and documents.
- Loads `BM25` results (from Assignment 1).
- Encodes query and document texts into embeddings.
- Computes cosine similarity between queries and documents.
- Combines neural scores with baseline `BM25` scores using a hybrid scoring formula.
- Outputs re-ranked top-100 results per query using both `BERT` and `USE`.

For our experiment, we implemented a hybrid re-ranking approach using two pre-trained models: **BERT** from the Sentence-Transformers library and the **Universal Sentence Encoder (USE)** from TensorFlow Hub. These models were chosen for their strong semantic encoding capabilities, allowing us to represent both queries and documents as dense vector embeddings. We first loaded the **BM25 results from Assignment 1**, which provided the top 100 candidate documents per query. Each document and query was then embedded using both BERT and USE separately. The BERT model we used was `"bert-base-nli-mean-tokens"`, while for USE we used the official TensorFlow Hub model `"https://tfhub.dev/google/universal-sentence-encoder/4"`. We precomputed document embeddings for efficiency and stored them in dictionaries, allowing for fast lookup during the re-ranking process.

To compute similarity, we used cosine similarity between query and document embeddings. These similarity scores were then normalized and combined with the baseline BM25 scores from Assignment 1 using a hybrid formula: `final_score = alpha * neural_score + (1 - alpha) * baseline_score`, with alpha set to 0.35. This method allowed us to balance the lexical matching strength of BM25 with the semantic understanding from neural models. We processed all queries in parallel using multithreading to speed up computation. The final re-ranked results were saved in two output files: `Results_BERT.txt` and `Results_USE.txt`. These files followed the TREC format and contained the top 100 ranked documents per query for each embedding model.

Folder Layout

```
A2/
├── dataset/
│   ├── corpus.jsonl          # Corpus file
│   ├── queries.jsonl        # Queries file
│   └── qrels/
│       └── test.tsv
├── Results_A1_BM25.txt       # Baseline output from Assignment 1 BM25
├── models/
│   ├── bert_model.py         # BERT embedder module
│   └── use_model.py          # USE embedder module
├── neural_ranking.py         # Neural re-ranking functions using BERT and USE
├── Results_BERT.txt          # Output result file BERT
├── Results_USE.txt           # Output result file USE
└── main.py                   # Main driver that reads baseline results and
                             applies neural re-ranking
```

How to run the program

1. Dependencies:

- Install **Python 3.11**:
<https://www.python.org/downloads/release/python-3110/>
- Install **Sentence Transformers**: `pip3 install sentence-transformers`
- Install **Torch**: `pip install torch`
- Install **NumPy**: `pip install numpy`
- Install **TensorFlow Hub**: `pip install tensorflow tensorflow-hub`

Ensure `corpus.jsonl` and `queries.jsonl` are in the `dataset/` folder.

2. Execution: Run the main script:

```
python main.py
```

This will output ranking results to `Results_BERT.txt` for BERT results and `Results_USE.txt` for Universal sentence encoder results.

Algorithms, Data structures, and Optimizations

Algorithms:

We used a **hybrid re-ranking algorithm** that combines traditional lexical retrieval (BM25) with neural semantic matching (BERT and USE). The algorithm involves:

1. Cosine Similarity Calculation:

For each query-document pair, we calculate cosine similarity between their embedding vectors to measure semantic similarity. This is done using a vectorized computation to improve speed:

```
similarity = dot(query, doc) / (||query|| * ||doc||)
```

2. Score Normalization and Hybrid Scoring:

Both BM25 and neural scores are normalized to the range [0, 1] to ensure fair combination. The final hybrid score is computed as:

```
hybrid_score = alpha * neural_score + (1 - alpha) * baseline_score
```

where alpha is set to 0.35 to give moderate weight to the neural model.

Data Structures:

We used the following data structures to support efficient retrieval and computation:

- **Dictionaries (HashMaps):**

- `doc_map`: Maps `doc_id` to document content.
- `query_map`: Maps `query_id` to query text.
- `bert_embedding_dict` / `use_embedding_dict`: Maps `doc_id` to its precomputed embedding (for BERT or USE).
- `baseline_results`: Maps `query_id` to a list of tuples containing BM25 ranked results.

- **Lists:**
 - Used to store candidate documents, embeddings, and final ranked results.
 - **NumPy Arrays:**
 - Used for fast vector operations, especially for storing and manipulating embeddings during similarity computation and normalization.
-

Optimizations:

1. Precomputation of Document Embeddings:

Instead of encoding documents for every query, we encode all candidate documents once and store their embeddings in a dictionary. This significantly reduces redundant computation.

2. Vectorized Cosine Similarity:

Instead of computing cosine similarity in a loop, we use NumPy to compute all similarities in one go using matrix operations. This improves performance by leveraging optimized low-level libraries.

3. Multithreading with ThreadPoolExecutor:

We parallelized query processing using Python's `ThreadPoolExecutor` with multiple workers. This allows multiple queries to be processed simultaneously, making full use of available CPU cores.

4. Efficient Sorting and Output:

After computing scores, we sort results using Python's built-in `sort()` with a custom key for performance and write only the top 100 ranked results per query to output files in TREC format.

Sample queries

BERT:

```
1 Q0 21257564 1 0.8089 run_bert
1 Q0 7581911 2 0.8086 run_bert
1 Q0 36480032 3 0.7295 run_bert
1 Q0 20155713 4 0.7209 run_bert
1 Q0 26071782 5 0.6201 run_bert
1 Q0 18953920 6 0.6087 run_bert
1 Q0 10906636 7 0.5372 run_bert
1 Q0 13231899 8 0.5125 run_bert
1 Q0 994800 9 0.4831 run_bert
1 Q0 21456232 10 0.4787 run_bert
```

```
3 Q0 4414547 1 0.9300 run_bert
3 Q0 14717500 2 0.7544 run_bert
3 Q0 2739854 3 0.7239 run_bert
3 Q0 19058822 4 0.6513 run_bert
3 Q0 4378885 5 0.6374 run_bert
3 Q0 4632921 6 0.6198 run_bert
3 Q0 23389795 7 0.6183 run_bert
3 Q0 1388704 8 0.5693 run_bert
3 Q0 2107238 9 0.5437 run_bert
3 Q0 19497526 10 0.5297 run_bert
```

Universal Sentence-transformers:

```
1 Q0 21257564 1 0.8676 run_use
1 Q0 20155713 2 0.6378 run_use
1 Q0 26071782 3 0.6352 run_use
1 Q0 7581911 4 0.6206 run_use
1 Q0 36480032 5 0.6143 run_use
1 Q0 18953920 6 0.5264 run_use
1 Q0 3845894 7 0.5193 run_use
1 Q0 994800 8 0.5184 run_use
1 Q0 10906636 9 0.5137 run_use
1 Q0 13231899 10 0.5075 run_use
```

```
3 Q0 4414547 1 0.9554 run_use
3 Q0 2739854 2 0.8419 run_use
3 Q0 23389795 3 0.8015 run_use
3 Q0 4378885 4 0.7396 run_use
3 Q0 4632921 5 0.7259 run_use
3 Q0 14717500 6 0.7232 run_use
3 Q0 19058822 7 0.6286 run_use
3 Q0 461550 8 0.5896 run_use
3 Q0 2485101 9 0.5236 run_use
3 Q0 32181055 10 0.5186 run_use
```

Result

Mean Average Precision (MAP) and P@10 Score

The `MAP` and `P@10` score is computed using `trec_eval` based on the results from the `Results_BERT.txt` and `Results_USE.txt` file. Please make sure to install `trec_eval` and use `make` command to build the executable for MAP.

These were our `MAP` and `P@10` results for `Results_BERT.txt`:

map	all	0.5452
P_10	all	0.0800

These were our `MAP` and `P@10` results for `Results_USE.txt`:

map	all	0.5535
P_10	all	0.0800

These were our MAP and P@10 results from Assignment 1:

map	all	0.5337
P_10	all	0.0787

Both the BERT and USE hybrid models improved compared to the BM25 benchmark. the USE model had the highest MAP value (0.5535), which was slightly higher than BERT (0.5452) and BM25 (0.5337). Although the improvement in P@10 is small (from 0.0787 to 0.0800), the improvement in MAP suggests that the neural reordering strategy retrieved more relevant documents overall, even if it did not always retrieve the top 10.

This suggests that combining semantic representations from pre-trained neural models with traditional lexical scoring (via hybrid scoring) can be more effective for document ranking. The USE model seems to be slightly better at generalization for our dataset and query set, possibly due to the fact that it was trained on a wider range of sentence-level tasks.

In conclusion, the hybrid approach succeeds in improving retrieval quality without replacing BM25, demonstrating the benefits of combining lexical and semantic matching techniques.